FROM PIXELS TO PROVENANCE: HARNESSING SOURCE CAMERA FINGERPRINTS TO DETECT AI-CREATED IMAGES

ABSTRACT:

The rapid advancement of generative artificial intelligence has led to the widespread creation of highly realistic synthetic images, posing significant challenges to image authenticity, digital trust, and misinformation detection. Traditional visual inspection and content-based forensic techniques are increasingly inadequate in distinguishing AI-generated images from those captured by real-world cameras. This study presents a robust forensic framework that leverages source camera fingerprints to detect AI-created images by analyzing intrinsic device-level artifacts embedded during the image formation process. By extracting and modeling sensor-specific patterns such as Photo-Response Non-Uniformity (PRNU) and noise residuals, the proposed approach differentiates between images originating from physical imaging sensors and those synthesized by generative models. The framework integrates signal processing techniques with machine learning classifiers to evaluate provenance authenticity at the pixel level. Experimental results demonstrate that camera fingerprint–based analysis significantly improves detection accuracy, even for visually convincing AI-generated images and post-processed content. The findings highlight the effectiveness of provenance-aware forensics in combating deepfake imagery and reinforcing trust in digital media ecosystems. This work contributes toward the development of reliable image authentication systems essential for digital forensics, journalism, legal evidence verification, and secure multimedia communication.

Keywords: AI-Generated Images, Source Camera Fingerprints, PRNU, Image Provenance, Digital Image Forensics, Deepfake Detection, Multimedia Security.

1. INTRODUCTION

1.1 Introduction

In recent years, the rapid evolution of artificial intelligence, particularly deep learning–based generative models, has transformed the way digital images are created and consumed. Advanced generative techniques such as Generative Adversarial Networks (GANs) and diffusion models are capable of producing highly realistic images that are often indistinguishable from photographs captured by real-world cameras. While these technologies offer significant benefits in areas such as content creation, entertainment, and design, they also introduce serious challenges related to image authenticity, misinformation, copyright infringement, and digital trust.

Images have traditionally been regarded as reliable visual evidence in journalism, legal proceedings, scientific research, and social media communication. However, the emergence of AI-generated images has weakened this trust, as synthetic content can be easily misused to fabricate events, manipulate public opinion, or spread false information. Conventional image forensics techniques that rely on visual cues, metadata analysis, or manual inspection are increasingly ineffective against sophisticated AI-generated imagery, which often lacks obvious artifacts.

Digital image forensics has therefore shifted toward more intrinsic and device-level analysis methods to verify image provenance. One promising direction is the use of source camera fingerprints, which exploit unique sensor imperfections introduced during the image acquisition process. These fingerprints act as a distinctive signature of a physical camera, enabling forensic analysts to determine whether an image originated from a real imaging sensor or was artificially generated. This chapter introduces the fundamental

motivation, objectives, scope, and necessity of leveraging source camera fingerprints to detect AI-created images and ensure trustworthy digital media.

1.2 Motivation

The primary motivation for this study arises from the growing misuse of AI-generated images in misinformation campaigns, deepfake creation, and digital fraud. As generative models continue to improve, synthetic images are becoming visually flawless, making it increasingly difficult for humans and traditional automated systems to distinguish them from authentic photographs. This poses a significant threat to digital ecosystems that rely on image-based evidence.

Another motivating factor is the limitation of existing detection approaches that focus mainly on content-level features or statistical artifacts specific to known generative models. Such methods often fail when images are post-processed, compressed, or generated using unseen models. In contrast, source camera fingerprint analysis focuses on the physical characteristics of camera sensors, which AI-generated images inherently lack. This shift from visual realism to provenance verification provides a more robust and future-proof solution.

Furthermore, the increasing reliance on digital images in legal, journalistic, and security-sensitive domains demands reliable forensic tools that can support automated, scalable, and objective decision-making. A system capable of identifying the origin of an image at the sensor level can assist investigators, journalists, and platform moderators in combating misinformation and restoring confidence in digital content.

1.3 Aim

The main aim of this study is to design and develop a robust forensic framework that utilizes source camera fingerprints to detect and distinguish AI-generated images from those captured by real-world cameras. The proposed approach seeks to analyze intrinsic sensor-level artifacts embedded in images to establish their provenance and authenticity.

1.4 Scope

The scope of this study is focused on digital image forensics and provenance analysis using source camera fingerprints. The research involves analyzing noise-based sensor patterns, such as Photo-Response Non-Uniformity (PRNU), and employing machine learning techniques to classify images as camera-captured or AI-generated.

The scope includes:

Analysis of intrinsic camera sensor fingerprints

Extraction of noise residuals and device-level features

Application of machine learning classifiers for image source detection

Evaluation of system performance using standard forensic metrics

The study does not involve real-time video analysis or biometric identification. However, the proposed framework can be extended in future work to support video forensics, cross-device analysis, and large-scale social media monitoring systems.

1.5 Objectives

The specific objectives of this study are as follows:

To study the fundamentals of AI-generated image synthesis and its forensic challenges

To analyze source camera fingerprints and sensor noise characteristics

To extract discriminative features that differentiate real and AI-generated images
To implement machine learning models for image provenance classification
To evaluate the effectiveness of camera fingerprint–based detection methods
To develop a reliable forensic framework that supports digital trust and authenticity

1.6 Problem Statement

With the rapid proliferation of AI-generated images, ensuring the authenticity and origin of digital imagery has become a critical challenge. Existing forensic techniques that rely on visual artifacts, metadata, or model-specific traces are increasingly unreliable due to advancements in generative models and image post-processing techniques. As a result, there is no universally reliable method to determine whether an image originates from a physical camera or is synthetically generated.

The problem addressed in this study is the lack of a robust, provenance-aware detection mechanism that can reliably identify AI-created images by analyzing intrinsic sensor-level characteristics rather than superficial visual features. Addressing this problem is essential for maintaining trust in digital media and preventing the misuse of synthetic imagery.

1.7 Need of the Study

The need for this study is driven by the growing dependence on digital images as sources of information and evidence in modern society. The increasing realism of AI-generated images threatens the credibility of visual media, leading to potential social, legal, and ethical consequences. Without reliable detection mechanisms, false imagery can undermine public trust, manipulate opinions, and compromise judicial processes.

From a technological perspective, this study contributes to the advancement of digital forensics by emphasizing provenance-based analysis over appearance-based detection. By leveraging source camera fingerprints, the proposed approach offers a more resilient solution against evolving generative models.

Moreover, the study supports the development of intelligent forensic systems that assist human experts rather than replace them. Such systems can be integrated into content verification platforms, social media moderation tools, and legal investigation workflows, ultimately helping to preserve the integrity, authenticity, and trustworthiness of digital images.

.

Chapter -2
LITERATURE SURVEY

he rapid growth of artificial intelligence–driven image generation has prompted extensive research in the field of digital image forensics, with a particular focus on detecting synthetic and manipulated visual content. This chapter presents a comprehensive review of existing studies related to AI-generated image detection, source camera fingerprint analysis, and image provenance verification. The survey highlights key methodologies, findings, and limitations of prior research, providing a foundation for the proposed approach.

Several early studies in digital image forensics concentrated on identifying image tampering using visual inconsistencies, compression artifacts, and metadata analysis. Researchers such as Farid emphasized forensic techniques based on statistical irregularities introduced during image editing and manipulation. While these methods proved effective for traditional image forgeries, they were primarily designed to detect post-processing operations such as splicing, resampling, and compression. With the emergence of AI-generated imagery, these approaches became less reliable due to the absence of conventional manipulation traces.

The introduction of Generative Adversarial Networks (GANs) marked a significant turning point in synthetic image creation. Studies by Goodfellow et al. demonstrated that GANs could generate highly realistic images by learning complex data distributions. Subsequent research focused on detecting GAN-generated images using frequency-domain analysis, color inconsistencies, and deep learning–based classifiers. Although these techniques achieved promising results, their effectiveness often diminished when images were generated using unseen architectures or subjected to post-processing operations.

To address the limitations of content-based detection, researchers began exploring intrinsic image properties related to the image acquisition process. One of the most influential contributions in this area is the concept of Photo-Response Non-Uniformity (PRNU), introduced by Lukas, Fridrich, and Goljan. PRNU represents a unique sensor-level noise pattern caused by manufacturing imperfections in camera sensors. Numerous studies have shown that PRNU can be used to reliably identify the source camera of an image, even after moderate image processing.

Further research expanded PRNU-based techniques to camera identification, forgery localization, and image authentication. Chen et al. demonstrated the robustness of sensor pattern noise for camera attribution across different imaging conditions. These studies established source camera fingerprints as a powerful forensic tool, particularly for verifying the authenticity and provenance of digital images. However, most of this work focused on images captured by physical cameras and did not consider AI-generated content.

With the increasing prevalence of synthetic imagery, recent studies have investigated the absence or inconsistency of camera fingerprints in AI-generated images. Researchers such as Cozzolino and Verdoliva analyzed noise residuals in synthetic images and observed that AI-generated content lacks the structured sensor noise present in real photographs. Their findings suggested that the absence of PRNU-like patterns could serve as a strong indicator of synthetic origin.

Other researchers proposed hybrid approaches that combine signal processing techniques with machine learning models to enhance detection performance. These methods extract noise-based features and feed them into classifiers such as Support Vector Machines, Random Forests, or Convolutional Neural Networks. Experimental results indicated improved generalization compared to purely content-based detectors. Nevertheless, challenges remain in handling low-quality images, aggressive compression, and images generated by advanced diffusion models.

More recent literature has explored provenance-aware forensics, emphasizing the importance of determining the origin and creation history of digital content. Studies in this domain argue that future forensic systems must move beyond visual realism and focus on intrinsic generation traces that are difficult to replicate artificially. Source camera fingerprint analysis aligns closely with this vision, as it leverages physical-world constraints that AI-generated images cannot naturally emulate.

In summary, the literature reveals a clear evolution from traditional manipulation detection to advanced provenance-based analysis. While existing approaches for AI-generated image detection offer valuable insights, many suffer from limited robustness and poor generalization. Source camera fingerprint–based techniques emerge as a promising and reliable alternative, motivating the proposed framework that harnesses sensor-level artifacts to distinguish AI-created images from genuine camera-captured photographs.

Chapter 3
SYSTEM DESIGN & ANALYSIS

System design and analysis involve a detailed examination of existing approaches for detecting AI-generated images and evaluating their limitations. Based on this analysis, an improved forensic framework is proposed that leverages source camera fingerprints to establish image provenance. This chapter presents a comparative discussion of the existing system and the proposed system, outlining their methodologies, shortcomings, and advantages.

3.1 Existing System

Existing systems for detecting AI-generated or manipulated images predominantly rely on content-based forensic techniques and deep learning–based visual analysis. These approaches examine pixel-level inconsistencies, frequency-domain artifacts, color distribution anomalies, and semantic irregularities introduced during image synthesis. Many systems employ convolutional neural networks trained on large datasets of real and synthetic images to classify image authenticity.

In addition, some methods utilize metadata analysis, watermark detection, or heuristic rules to identify manipulated images. While these techniques can be effective under controlled conditions, they often depend heavily on known generative models and training datasets. As generative architectures evolve rapidly, such systems struggle to generalize to unseen AI models or novel image synthesis techniques.

Another limitation of existing systems is their vulnerability to post-processing operations such as compression, resizing, filtering, and noise addition. These operations can significantly degrade or remove detectable artifacts, leading to reduced detection accuracy. Furthermore, most current solutions focus on identifying visual realism rather than determining the true origin of an image, making them reactive rather than provenance-aware.

Overall, existing systems are limited in robustness, scalability, and long-term reliability, especially in real-world scenarios where image quality varies and adversarial techniques are employed.

Disadvantages of Existing System

- Limited generalization to unseen AI-generated image models
- High dependency on visual and content-based artifacts
- Reduced accuracy under image post-processing and compression
- Lack of provenance-level verification
- Susceptibility to adversarial manipulation
- Poor scalability for large-scale image verification

3.2 Proposed System

The proposed system introduces a provenance-aware forensic framework designed to detect AI-created images by analyzing source camera fingerprints. Unlike conventional content-based approaches, this system focuses on intrinsic sensor-level characteristics that are naturally present in camera-captured images but absent in AI-generated content.

In the proposed approach, images undergo preprocessing to extract noise residuals that contain sensor-specific artifacts. Techniques such as denoising filters and high-pass operations are applied to isolate Photo-

Response Non-Uniformity (PRNU) and related fingerprint patterns. These extracted features serve as a reliable representation of a physical camera's unique signature.

The system integrates machine learning classifiers to distinguish between images containing authentic camera fingerprints and those lacking such characteristics. By learning discriminative patterns from sensor noise features, the model can effectively classify images as real or AI-generated, even when visual quality is high or post-processing is applied.

The proposed framework is designed to be modular and scalable, enabling integration into digital forensic tools, content moderation platforms, and image verification pipelines. By shifting the focus from visual appearance to provenance verification, the system supports proactive and reliable detection of synthetic imagery.

Advantages of Proposed System

- Provenance-based detection using intrinsic sensor fingerprints
- High robustness against unseen AI models
- Improved resistance to post-processing operations
- Reduced reliance on visual artifacts
- Scalable and automated forensic analysis
- Enhanced reliability and long-term applicability

Chapter – 4
SYSTEM SPECIFICATION

4.1 System Requirements

The system requirements define the necessary hardware and software resources required for the effective implementation of the proposed AI-generated image detection framework. The system is designed to operate efficiently using standard computing resources, making it suitable for deployment in academic, forensic, and institutional environments.

HARDWARE REQUIREMENTS

System : Intel Pentium IV / Equivalent Processor (2.4 GHz or higher)
Hard Disk : 40 GB or above
RAM : 512 MB or above
Monitor : 14" Color Monitor
Mouse : Optical Mouse
Keyboard : Standard Keyboard

SOFTWARE REQUIREMENTS

Operating System : Windows 7 / Windows 10 / Linux
Programming Language : Python
Front-End : HTML, CSS, JavaScript
Framework : Django
Database : MySQL / SQLite
Libraries & Tools : NumPy, OpenCV, Scikit-learn, TensorFlow / PyTorch

4.2 Functional and Non-Functional Requirements

System requirements are further classified into functional and non-functional requirements to define what the system should do and how it should perform.

4.2.1 Functional Requirements

Functional requirements describe the core functionalities that the system must support.
The system shall allow users to upload digital images for analysis.
The system shall preprocess input images to extract noise residuals.
The system shall extract source camera fingerprints from images.
The system shall apply machine learning models for image classification.
The system shall identify whether an image is camera-captured or AI-generated.
The system shall display classification results clearly to the user.
The system shall store analysis results for future reference (optional).
The system shall support multiple image formats.

4.2.2 Non-Functional Requirements

Non-functional requirements define system performance, reliability, and usability characteristics.

- Performance: The system should provide results with minimal processing delay.
- Accuracy: The system should achieve high classification accuracy.
- Scalability: The system should efficiently handle large image datasets.
- Reliability: The system should produce consistent and dependable results.
- Usability: The system should be user-friendly and easy to operate.
- Security: Uploaded images and results should be handled securely.

- Maintainability: The system should allow easy updates and enhancements.
- Portability: The system should be deployable across different platforms.

4.3 Study of System (Feasibility Study)

The study of the system evaluates the feasibility and stability of the proposed provenance-aware image detection framework.

4.3.1 Technical Feasibility

The proposed system is technically feasible as it relies on established image processing and machine learning techniques. Python-based libraries provide robust support for sensor fingerprint extraction, feature analysis, and classification. The system does not require specialized hardware, making it practical for real-world deployment.

4.3.2 Economic Feasibility

The system is economically feasible since it utilizes open-source tools and software frameworks. Deployment on standard hardware reduces implementation costs, while automated image verification minimizes manual forensic effort and associated expenses.

4.3.3 Operational Feasibility

The system is operationally feasible as it is designed for ease of use by forensic analysts, researchers, and content moderators. Minimal training is required to operate the system, and its workflow integrates smoothly with existing digital forensic processes.

4.3.4 Stability and Reliability

The system ensures stable and reliable operation by leveraging validated algorithms and structured processing pipelines. Regular updates to the classification models can further enhance detection accuracy and maintain long-term reliability.

Chapter-5
SYSTEM DESIGN

UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1.Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.

2.Provide extendibility and specialization mechanisms to extend the core concepts.

3.Be independent of particular programming languages and development process.

4.Provide a formal basis for understanding the modeling language.

5.Encourage the growth of OO tools market.

6.Support higher level development concepts such as collaborations, frameworks, patterns and components.

7.Integrate best practices.

Module

1. Remote User Module

This module handles the interaction for the end-user who needs to verify images.

- User Registration: Allows new users to create an account by providing 6 fields (e.g., username, email, password, address, phone, and profile info).
- User Login: Secure authentication using SQLite3 to verify authorized credentials.
- View Profile: Users can manage and view their personal registration details.
- Data Management: Browse, upload, and manage the training datasets.
- Model Training: Execute the training process on the dataset to improve detection accuracy.
- Performance Analytics:
- Table Format: View detailed accuracy results and error rates in a structured table.
- Graph Format: Generate bar charts and ratios to visualize trained vs. tested accuracy.
- AI Detection (Testing): The core interface where a user uploads a single image to detect if it is Real (Human Created) or Fake (AI Generated).

2. Image Processing & Detection Pipeline

This technical module contains the logic for analyzing the images as defined in your architecture.

A. Preprocessing

- Resize: Standardizing image dimensions for the neural network.
- Normalize: Scaling pixel values for faster convergence.
- Color to Gray: Converting images to grayscale where color is not a primary feature for noise analysis.

B. Feature Extraction

- Noise Residual Extraction: Using High-pass or Wavelet filters to isolate the "fingerprint" left by AI generators.
- Fingerprint Feature Extraction: Utilizing CNN / Frequency CNN and Global Pooling to identify patterns unique to GANs or Diffusion models.

C. Classifier & Evaluation

- Classifier: Determines if the image is Real vs. AI and optionally identifies which generator was used.
- Evaluation: Performs cross-model testing and checks for post-processing robustness (e.g., can the model still detect a fake if the image was compressed or blurred?).

3. Database Schema (SQLite3)

Table	Purpose
Users	Stores the 6 registration fields and authorization status.
Training_Log	Stores accuracy results, loss values, and training timestamps.
Detection_History	Logs the results of tested images (Real vs. Fake) for user history.

SYSTEM ARCHITECTURE:

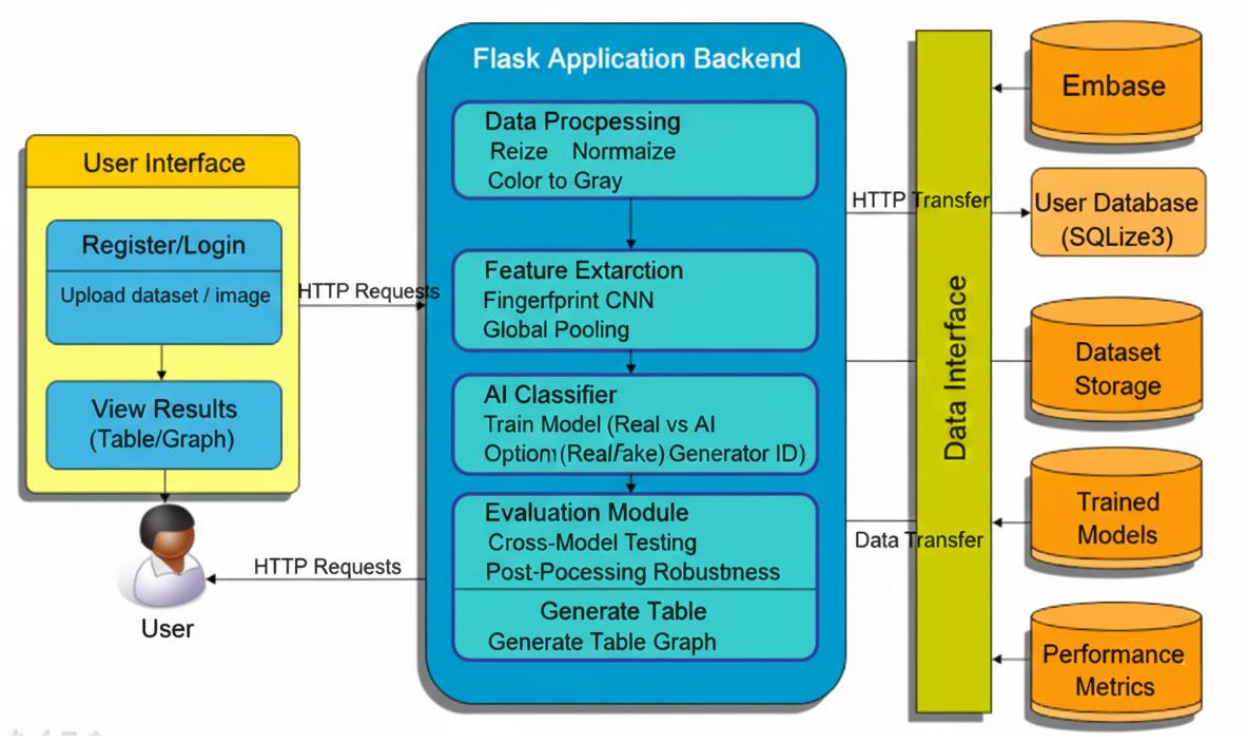

Fig : System Architecture

DATA FLOW DIAGRAM:

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail

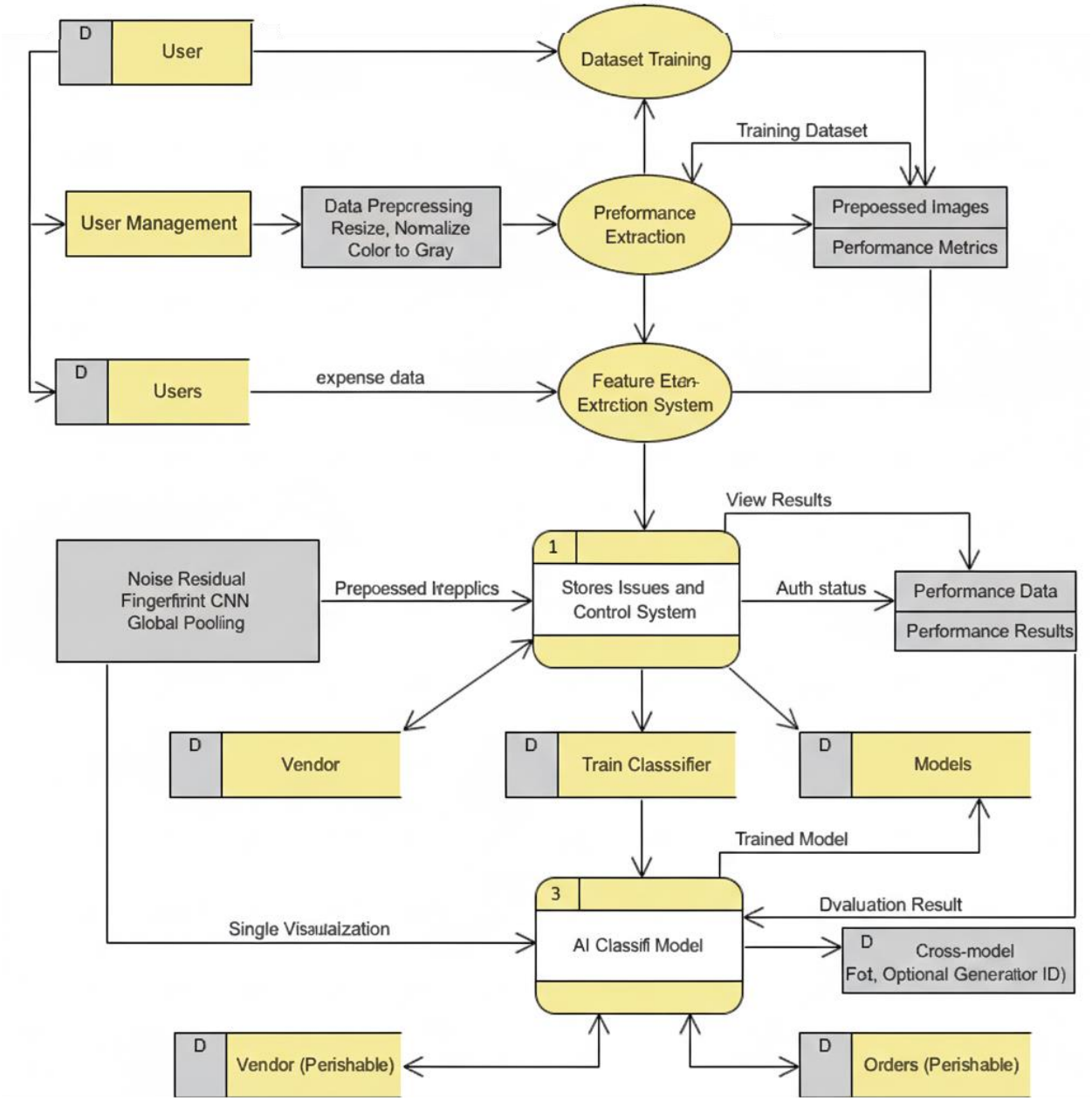

Fig: Data Flow Diagram

USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

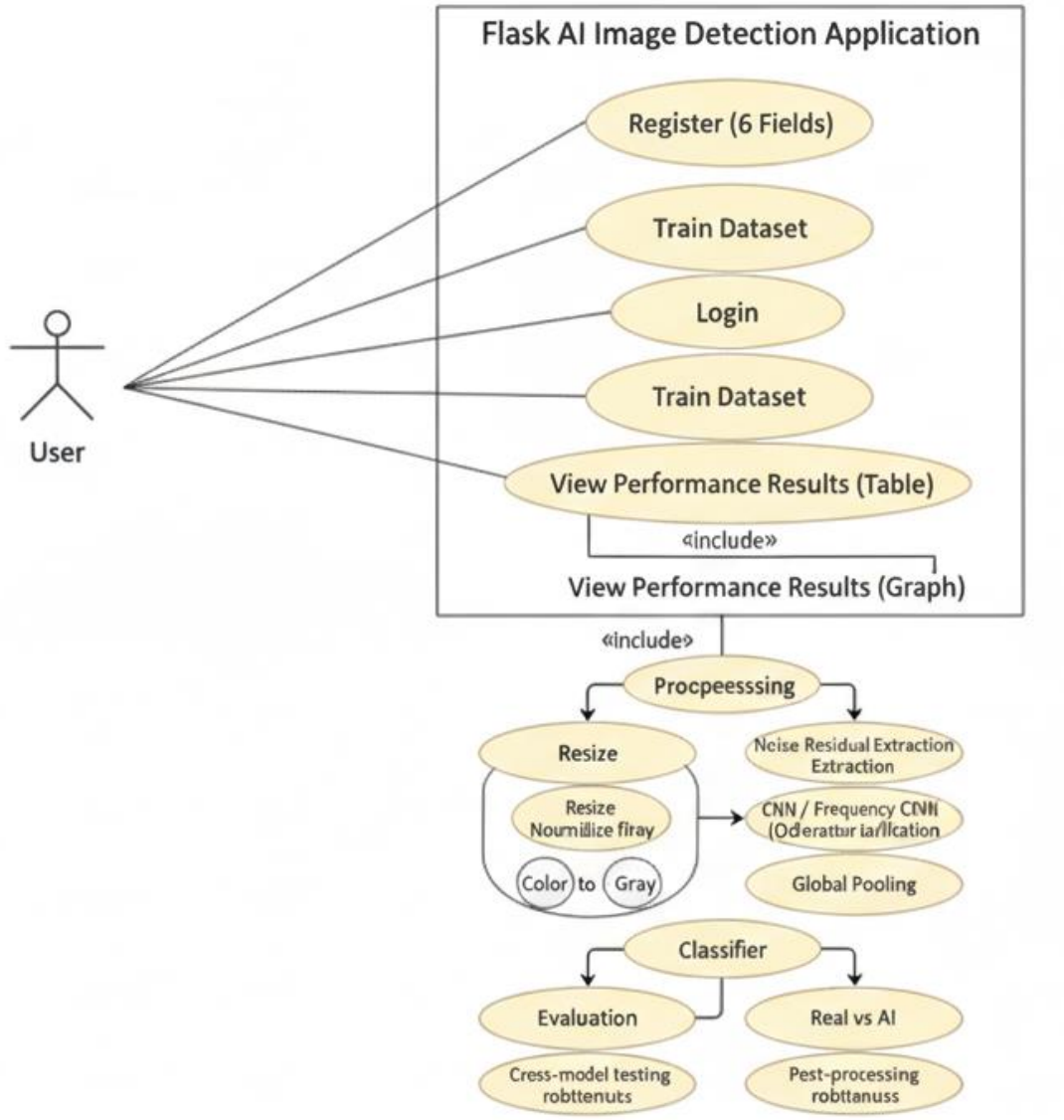

Fig: Use Case Diagram

CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information

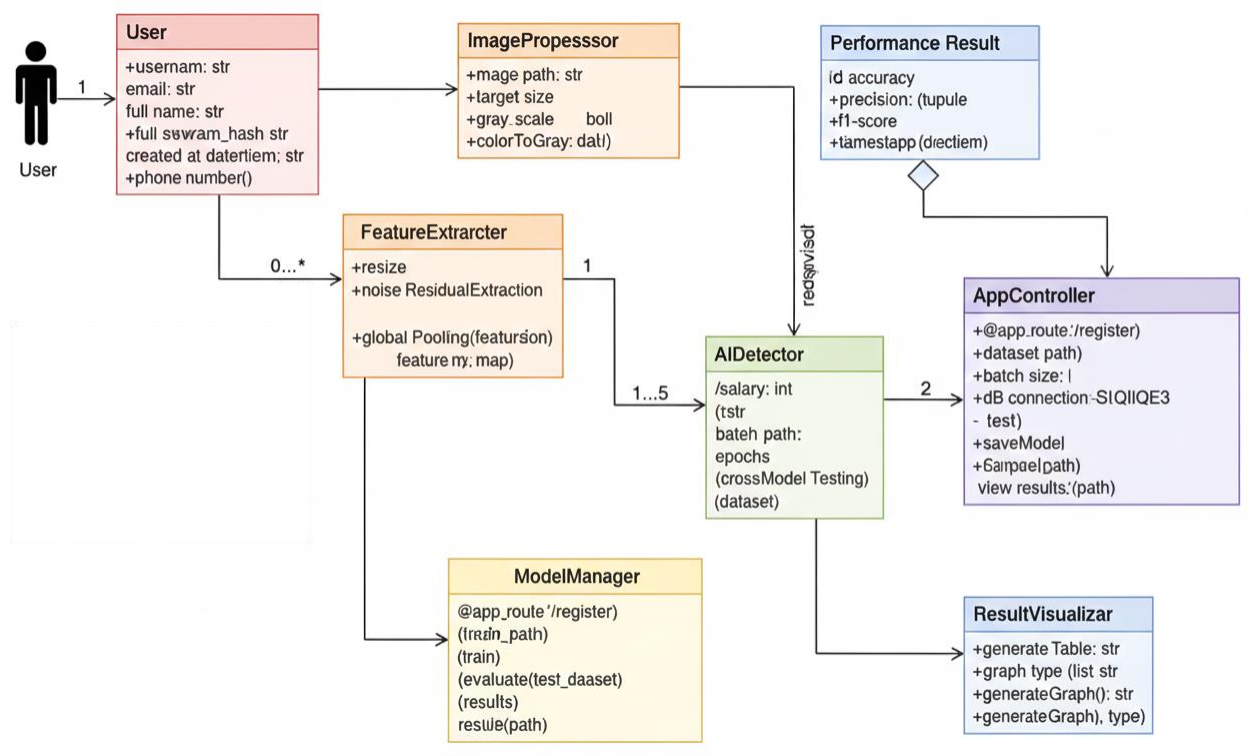

Fig: Class Diagram

SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

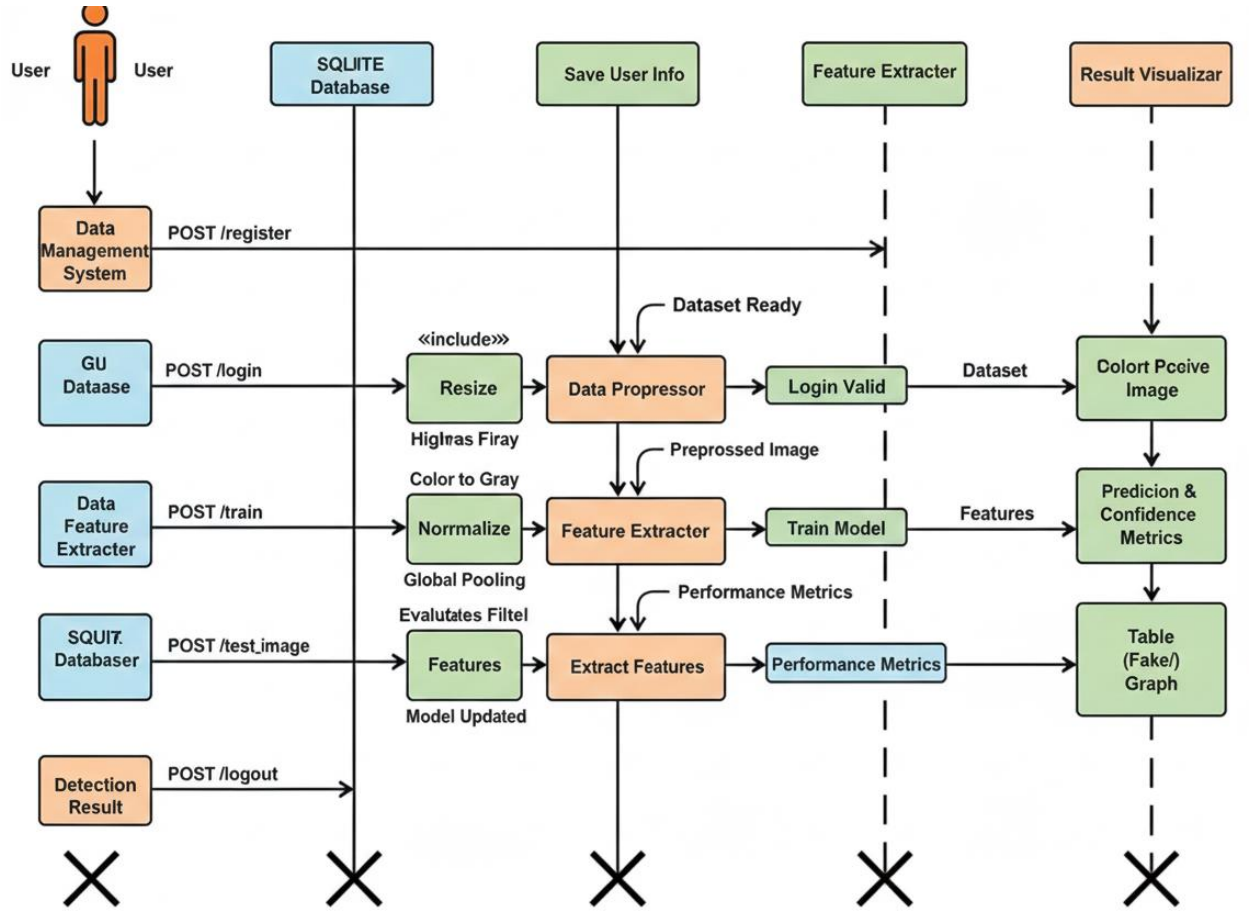

Fig: Sequence Diagram

ACTIVITY DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control

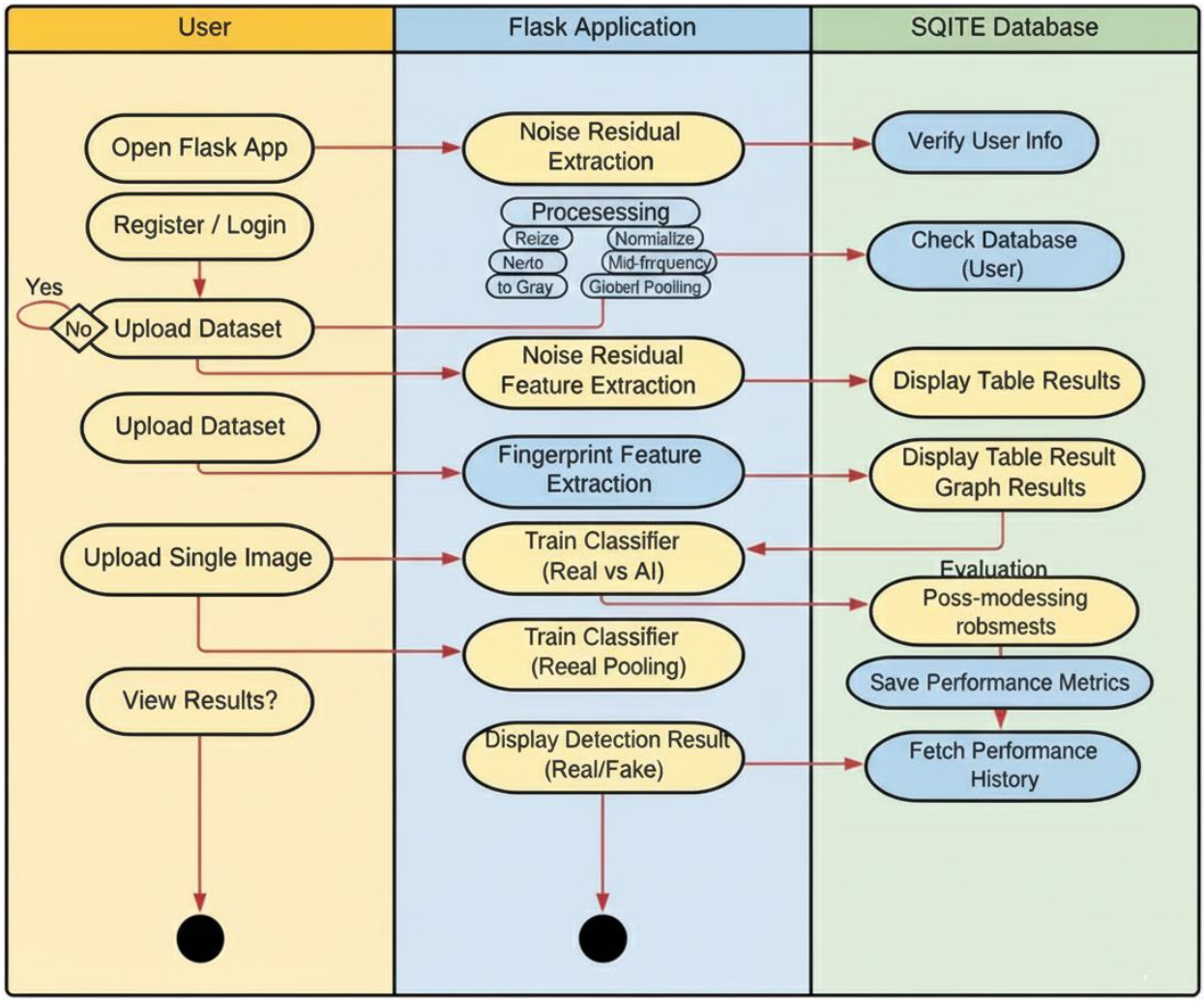

Fig: Activity Diagram

Chapter-6
Software Environment

PYTHON

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. An interpreted language, Python has a design philosophy that emphasizes code readability (notably using whitespace indentation to delimit code blocks rather than curly brackets or keywords), and a syntax that allows programmers to express concepts in fewer lines of code than might be used in languages such as C++or Java. It provides constructs that enable clear programming on both small and large scales. Python interpreters are available for many operating systems. CPython, the reference implementation of Python, is open source software and has a community-based development model, as do nearly all of its variant implementations. CPython is managed by the non-profit Python Software Foundation. Python features a dynamic type system and

automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

Interactive Mode Programming

Invoking the interpreter without passing a script file as a parameter brings up the following prompt −

```
$ python
Python 2.4.3 (#1, Nov 11 2010, 13:34:43)
[GCC 4.1.2 20080704 (Red Hat 4.1.2-48)] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Type the following text at the Python prompt and press the Enter −

```
>>> print "Hello, Python!"
```

If you are running new version of Python, then you would need to use print statement with parenthesis as in print ("Hello, Python!");. However in Python version 2.4.3, this produces the following result −

```
Hello, Python!
```

Script Mode Programming

Invoking the interpreter with a script parameter begins execution of the script and continues until the script is finished. When the script is finished, the interpreter is no longer active.

Let us write a simple Python program in a script. Python files have extension .py. Type the following source code in a test.py file −

Live Demo

```
print "Hello, Python!"
```

We assume that you have Python interpreter set in PATH variable. Now, try to run this program as follows −

```
$ python test.py
```

This produces the following result −

```
Hello, Python!
```

Let us try another way to execute a Python script. Here is the modified test.py file −

Live Demo

```
#!/usr/bin/python

print "Hello, Python!"
```

We assume that you have Python interpreter available in /usr/bin directory. Now, try to run this program as follows −

```
$ chmod +x test.py     # This is to make file executable
$./test.py
```

This produces the following result −

Hello, Python!

Python Identifiers

A Python identifier is a name used to identify a variable, function, class, module or other object. An identifier starts with a letter A to Z or a to z or an underscore (_) followed by zero or more letters, underscores and digits (0 to 9).

Python does not allow punctuation characters such as @, $, and % within identifiers. Python is a case sensitive programming language. Thus, Manpower and manpower are two different identifiers in Python.

Here are naming conventions for Python identifiers −

Class names start with an uppercase letter. All other identifiers start with a lowercase letter.

Starting an identifier with a single leading underscore indicates that the identifier is private.

Starting an identifier with two leading underscores indicates a strongly private identifier.

If the identifier also ends with two trailing underscores, the identifier is a language-defined special name.

Reserved Words

The following list shows the Python keywords. These are reserved words and you cannot use them as constant or variable or any other identifier names. All the Python keywords contain lowercase letters only.

and	exec	not
assert	finally	or
break	for	pass
class	from	print
continue	global	raise
def	if	return
del	import	try
elif	in	while
else	is	with
except	lambda	yield

Lines and Indentation

Python provides no braces to indicate blocks of code for class and function definitions or flow control. Blocks of code are denoted by line indentation, which is rigidly enforced.

The number of spaces in the indentation is variable, but all statements within the block must be indented the same amount. For example −

```
if True:
   print "True"
else:
   print "False"
```

However, the following block generates an error −

```
if True:
print "Answer"
```

```
print "True"
else:
print "Answer"
print "False"
```

Thus, in Python all the continuous lines indented with same number of spaces would form a block. The following example has various statement blocks −

Note − Do not try to understand the logic at this point of time. Just make sure you understood various blocks even if they are without braces.

```
#!/usr/bin/python

import sys

try:
   # open file stream
   file = open(file_name, "w")
except IOError:
   print "There was an error writing to", file_name
   sys.exit()
print "Enter '", file_finish,
print "' When finished"
while file_text != file_finish:
   file_text = raw_input("Enter text: ")
   if file_text == file_finish:
      # close the file
      file.close
      break
   file.write(file_text)
   file.write("\n")
file.close()
file_name = raw_input("Enter filename: ")
if len(file_name) == 0:
   print "Next time please enter something"
   sys.exit()
try:
   file = open(file_name, "r")
except IOError:
   print "There was an error reading file"
   sys.exit()
file_text = file.read()
file.close()
print file_text
```

Multi-Line Statements

Statements in Python typically end with a new line. Python does, however, allow the use of the line continuation character (\) to denote that the line should continue. For example −

```
total = item_one + \
        item_two + \
        item_three
```

Statements contained within the [], {}, or () brackets do not need to use the line continuation character. For example −

```
days = ['Monday', 'Tuesday', 'Wednesday',
        'Thursday', 'Friday']
```

Quotation in Python

Python accepts single ('), double (") and triple ('" or """) quotes to denote string literals, as long as the same type of quote starts and ends the string.

The triple quotes are used to span the string across multiple lines. For example, all the following are legal −

```
word = 'word'
sentence = "This is a sentence."
paragraph = """This is a paragraph. It is
made up of multiple lines and sentences."""
```

Comments in Python

A hash sign (#) that is not inside a string literal begins a comment. All characters after the # and up to the end of the physical line are part of the comment and the Python interpreter ignores them.

Live Demo

```
#!/usr/bin/python

# First comment
print "Hello, Python!" # second comment
```

This produces the following result −

```
Hello, Python!
```

You can type a comment on the same line after a statement or expression −

```
name = "Madisetti" # This is again comment
```

You can comment multiple lines as follows −

```
# This is a comment.
# This is a comment, too.
# This is a comment, too.
# I said that already.
```

Following triple-quoted string is also ignored by Python interpreter and can be used as a multiline comments:

```
'''
This is a multiline
comment.
'''
```

Using Blank Lines

A line containing only whitespace, possibly with a comment, is known as a blank line and Python totally ignores it.

In an interactive interpreter session, you must enter an empty physical line to terminate a multiline statement.

Waiting for the User

The following line of the program displays the prompt, the statement saying “Press the enter key to exit”, and waits for the user to take action −

```
#!/usr/bin/python

raw_input("\n\nPress the enter key to exit.")
```

Here, "\n\n" is used to create two new lines before displaying the actual line. Once the user presses the key, the program ends. This is a nice trick to keep a console window open until the user is done with an application.

Multiple Statements on a Single Line

The semicolon (;) allows multiple statements on the single line given that neither statement starts a new code block. Here is a sample snip using the semicolon.

```
import sys; x = 'foo'; sys.stdout.write(x + '\n')
```

Multiple Statement Groups as Suites

A group of individual statements, which make a single code block are called suites in Python. Compound or complex statements, such as if, while, def, and class require a header line and a suite.

Header lines begin the statement (with the keyword) and terminate with a colon (:) and are followed by one or more lines which make up the suite. For example −

```
if expression :
   suite
elif expression :
   suite
else :
   suite
```

Command Line Arguments

Many programs can be run to provide you with some basic information about how they should be run. Python enables you to do this with -h −

```
$ python -h
```

usage: python [option] ... [-c cmd | -m mod | file | -] [arg] ...
Options and arguments (and corresponding environment variables):
-c cmd : program passed in as string (terminates option list)
-d : debug output from parser (also PYTHONDEBUG=x)
-E : ignore environment variables (such as PYTHONPATH)
-h : print this help message and exit
You can also program your script in such a way that it should accept various options. Command Line Arguments is an advanced topic and should be studied a bit later once you have gone through rest of the Python concepts.

Python Lists

The list is a most versatile datatype available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets. For example −

```
list1 = ['physics', 'chemistry', 1997, 2000];
list2 = [1, 2, 3, 4, 5 ];
list3 = ["a", "b", "c", "d"]
```

Similar to string indices, list indices start at 0, and lists can be sliced, concatenated and so on.

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally you can put these comma-separated values between parentheses also. For example −

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5 );
tup3 = "a", "b", "c", "d";
```

The empty tuple is written as two parentheses containing nothing −

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value −

```
tup1 = (50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

Accessing Values in Tuples

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example −

Live Demo

```
#!/usr/bin/python

tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5, 6, 7 );
print "tup1[0]: ", tup1[0];
print "tup2[1:5]: ", tup2[1:5];
```

When the above code is executed, it produces the following result −

```
tup1[0]:  physics
tup2[1:5]:  [2, 3, 4, 5]
```

Updating Tuples

Accessing Values in Dictionary

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example −

Live Demo

```
#!/usr/bin/python

dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
print "dict['Name']: ", dict['Name']
print "dict['Age']: ", dict['Age']
```

When the above code is executed, it produces the following result −

```
dict['Name']:  Zara
dict['Age']:  7
```

If we attempt to access a data item with a key, which is not part of the dictionary, we get an error as follows −

Live Demo

```
#!/usr/bin/python

dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
print "dict['Alice']: ", dict['Alice']
```

When the above code is executed, it produces the following result −

```
dict['Alice']:
Traceback (most recent call last):
  File "test.py", line 4, in <module>
    print "dict['Alice']: ", dict['Alice'];
KeyError: 'Alice'
```

Updating Dictionary

You can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example −

Live Demo

```
#!/usr/bin/python

dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
dict['Age'] = 8; # update existing entry
dict['School'] = "DPS School"; # Add new entry

print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

When the above code is executed, it produces the following result −

```
dict['Age']:  8
dict['School']:  DPS School
```

Delete Dictionary Elements

You can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the del statement. Following is a simple example −

Live Demo

```
#!/usr/bin/python

dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
del dict['Name']; # remove entry with key 'Name'
dict.clear();     # remove all entries in dict
del dict ;        # delete entire dictionary

print "dict['Age']: ", dict['Age']
print "dict['School']: ", dict['School']
```

This produces the following result. Note that an exception is raised because after del dict dictionary does not exist any more −

```
dict['Age']:
Traceback (most recent call last):
  File "test.py", line 8, in <module>
    print "dict['Age']: ", dict['Age'];
TypeError: 'type' object is unsubscriptable
```

Note − del() method is discussed in subsequent section.

Properties of Dictionary Keys

Dictionary values have no restrictions. They can be any arbitrary Python object, either standard objects or user-defined objects. However, same is not true for the keys.

There are two important points to remember about dictionary keys −

(a) More than one entry per key not allowed. Which means no duplicate key is allowed. When duplicate keys encountered during assignment, the last assignment wins. For example −

Live Demo

```
#!/usr/bin/python

dict = {'Name': 'Zara', 'Age': 7, 'Name': 'Manni'}
print "dict['Name']: ", dict['Name']
```

When the above code is executed, it produces the following result −

```
dict['Name']:  Manni
```

(b) Keys must be immutable. Which means you can use strings, numbers or tuples as dictionary keys but something like ['key'] is not allowed. Following is a simple example −

Live Demo

```
#!/usr/bin/python

dict = {['Name']: 'Zara', 'Age': 7}
print "dict['Name']: ", dict['Name']
```

When the above code is executed, it produces the following result −

```
Traceback (most recent call last):
  File "test.py", line 3, in <module>
    dict = {['Name']: 'Zara', 'Age': 7};
TypeError: unhashable type: 'list'
```

Tuples are immutable which means you cannot update or change the values of tuple elements. You are able to take portions of existing tuples to create new tuples as the following example demonstrates −

Live Demo

```
#!/usr/bin/python

tup1 = (12, 34.56);
tup2 = ('abc', 'xyz');
# Following action is not valid for tuples
# tup1[0] = 100;
# So let's create a new tuple as follows
tup3 = tup1 + tup2;
print tup3;
```

When the above code is executed, it produces the following result −

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the del statement. For example −

Live Demo

```
#!/usr/bin/python
tup = ('physics', 'chemistry', 1997, 2000);
print tup;
del tup;
print "After deleting tup : ";
print tup;
```

This produces the following result. Note an exception raised, this is because after del tup tuple does not exist any more −

```
('physics', 'chemistry', 1997, 2000)
After deleting tup :
Traceback (most recent call last):
  File "test.py", line 9, in <module>
    print tup;
NameError: name 'tup' is not defined
```

DJANGO

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It’s free and open source.

Django's primary goal is to ease the creation of complex, database-driven websites. Django emphasizes reusabilityand "pluggability" of components, rapid development, and the principle of don't repeat yourself. Python is used throughout, even for settings files and data models.

Django also provides an optional administrative create, read, update and delete interface that is generated dynamically through introspection and configured via admin models

Create a Project

Whether you are on Windows or Linux, just get a terminal or a cmd prompt and navigate to the place you want your project to be created, then use this code −

```
$ django-admin startproject myproject
```

This will create a "myproject" folder with the following structure −

```
myproject/
  manage.py
  myproject/
    __init__.py
    settings.py
    urls.py
    wsgi.py
```

The Project Structure

The “myproject” folder is just your project container, it actually contains two elements −

manage.py − This file is kind of your project local django-admin for interacting with your project via command line (start the development server, sync db...). To get a full list of command accessible via manage.py you can use the code −

$ python manage.py help

The “myproject” subfolder − This folder is the actual python package of your project. It contains four files −

__init__.py − Just for python, treat this folder as package.

settings.py − As the name indicates, your project settings.

urls.py − All links of your project and the function to call. A kind of ToC of your project.

wsgi.py − If you need to deploy your project over WSGI.

Setting Up Your Project

Your project is set up in the subfolder myproject/settings.py. Following are some important options you might need to set −

DEBUG = True

This option lets you set if your project is in debug mode or not. Debug mode lets you get more information about your project's error. Never set it to ‘True’ for a live project. However, this has to be set to ‘True’ if you want the Django light server to serve static files. Do it only in the development mode.

```
DATABASES = {
  'default': {
    'ENGINE': 'django.db.backends.sqlite3',
    'NAME': 'database.sql',
    'USER': '',
    'PASSWORD': '',
    'HOST': '',
    'PORT': '',
  }
}
```

Database is set in the ‘Database’ dictionary. The example above is for SQLite engine. As stated earlier, Django also supports −

MySQL (django.db.backends.mysql)

PostGreSQL (django.db.backends.postgresql_psycopg2)

Oracle (django.db.backends.oracle) and NoSQL DB

MongoDB (django_mongodb_engine)

Before setting any new engine, make sure you have the correct db driver installed.

You can also set others options like: TIME_ZONE, LANGUAGE_CODE, TEMPLATE…

Now that your project is created and configured make sure it's working −

$ python manage.py runserver

You will get something like the following on running the above code −

```
Validating models...
0 errors found
September 03, 2015 - 11:41:50
Django version 1.6.11, using settings 'myproject.settings'
Starting development server at http://127.0.0.1:8000/
```

Quit the server with CONTROL-C.

A project is a sum of many applications. Every application has an objective and can be reused into another project, like the contact form on a website can be an application, and can be reused for others. See it as a module of your project.

Create an Application

We assume you are in your project folder. In our main “myproject” folder, the same folder then manage.py −

```
$ python manage.py startapp myapp
```

You just created myapp application and like project, Django create a “myapp” folder with the application structure −

```
myapp/
  __init__.py
  admin.py
  models.py
  tests.py
  views.py
```

__init__.py − Just to make sure python handles this folder as a package.

admin.py − This file helps you make the app modifiable in the admin interface.

models.py − This is where all the application models are stored.

tests.py − This is where your unit tests are.

views.py − This is where your application views are.

Get the Project to Know About Your Application

At this stage we have our "myapp" application, now we need to register it with our Django project "myproject". To do so, update INSTALLED_APPS tuple in the settings.py file of your project (add your app name) −

```
INSTALLED_APPS = (
  'django.contrib.admin',
  'django.contrib.auth',
  'django.contrib.contenttypes',
  'django.contrib.sessions',
  'django.contrib.messages',
  'django.contrib.staticfiles',
  'myapp',
)
```

Creating forms in Django, is really similar to creating a model. Here again, we just need to inherit from Django class and the class attributes will be the form fields. Let's add a forms.py file in myapp folder to contain our app forms. We will create a login form.

myapp/forms.py

```
#-*- coding: utf-8 -*-
from django import forms
class LoginForm(forms.Form):
  user = forms.CharField(max_length = 100)
  password = forms.CharField(widget = forms.PasswordInput())
```

As seen above, the field type can take "widget" argument for html rendering; in our case, we want the password to be hidden, not displayed. Many others widget are present in Django: DateInput for dates, CheckboxInput for checkboxes, etc.

Using Form in a View

There are two kinds of HTTP requests, GET and POST. In Django, the request object passed as parameter to your view has an attribute called "method" where the type of the request is set, and all data passed via POST can be accessed via the request.POST dictionary.

Let's create a login view in our myapp/views.py −

```
#-*- coding: utf-8 -*-
from myapp.forms import LoginForm
def login(request):
  username = "not logged in"
  if request.method == "POST":
    #Get the posted form
    MyLoginForm = LoginForm(request.POST)
    if MyLoginForm.is_valid():
      username = MyLoginForm.cleaned_data['username']
  else:
    MyLoginForm = Loginform()
  return render(request, 'loggedin.html', {"username" : username})
```

The view will display the result of the login form posted through the loggedin.html. To test it, we will first need the login form template. Let's call it login.html.

```
<html>
  <body>

    <form name = "form" action = "{% url "myapp.views.login" %}"
      method = "POST" >{% csrf_token %}

      <div style = "max-width:470px;">
        <center>
          <input type = "text" style = "margin-left:20%;"
            placeholder = "Identifiant" name = "username" />
        </center>
      </div>

      <br>

      <div style = "max-width:470px;">
        <center>
          <input type = "password" style = "margin-left:20%;"
            placeholder = "password" name = "password" />
        </center>
      </div>
```

```
      <br>

      <div style = "max-width:470px;">
        <center>

          <button style = "border:0px; background-color:#4285F4; margin-top:8%;
            height:35px; width:80%;margin-left:19%;" type = "submit"
            value = "Login" >
            <strong>Login</strong>
          </button>

        </center>
      </div>

    </form>

  </body>
</html>
```

The template will display a login form and post the result to our login view above. You have probably noticed the tag in the template, which is just to prevent Cross-site Request Forgery (CSRF) attack on your site.

```
{% csrf_token %}
```

Once we have the login template, we need the loggedin.html template that will be rendered after form treatment.

```
<html>
  <body>
    You are : <strong>{{username}}</strong>
  </body>
</html>
```

Now, we just need our pair of URLs to get started: myapp/urls.py

```
from django.conf.urls import patterns, url
from django.views.generic import TemplateView
urlpatterns = patterns('myapp.views',
  url(r'^connection/',TemplateView.as_view(template_name = 'login.html')),
  url(r'^login/', 'login', name = 'login'))
```

When accessing "/myapp/connection", we will get the following login.html template rendered −

Setting Up Sessions

In Django, enabling session is done in your project settings.py, by adding some lines to the MIDDLEWARE_CLASSES and the INSTALLED_APPS options. This should be done while creating the project, but it's always good to know, so MIDDLEWARE_CLASSES should have −

'django.contrib.sessions.middleware.SessionMiddleware'

And INSTALLED_APPS should have −

'django.contrib.sessions'

By default, Django saves session information in database (django_session table or collection), but you can configure the engine to store information using other ways like: in file or in cache.

When session is enabled, every request (first argument of any view in Django) has a session (dict) attribute.

Let's create a simple sample to see how to create and save sessions. We have built a simple login system before (see Django form processing chapter and Django Cookies Handling chapter). Let us save the username in a cookie so, if not signed out, when accessing our login page you won't see the login form. Basically, let's make our login system we used in Django Cookies handling more secure, by saving cookies server side.

For this, first lets change our login view to save our username cookie server side

```
def login(request):
   username = 'not logged in'
   if request.method == 'POST':
      MyLoginForm = LoginForm(request.POST)
      if MyLoginForm.is_valid():
         username = MyLoginForm.cleaned_data['username']
         request.session['username'] = username
      else:
         MyLoginForm = LoginForm()
   return render(request, 'loggedin.html', {"username" : username}
```

Then let us create formView view for the login form, where we won't display the form if cookie is set _

```
def formView(request):
   if request.session.has_key('username'):
      username = request.session['username']
      return render(request, 'loggedin.html', {"username" : username})
   else:
      return render(request, 'login.html', {})
```

Now let us change the url.py file to change the url so it pairs with our new view

```
from django.conf.urls import patterns, url
from django.views.generic import TemplateView
urlpatterns = patterns('myapp.views',
   url(r'^connection/','formView', name = 'loginform'),
   url(r'^login/', 'login', name = 'login'))
```

When accessing /myapp/connection, you will get to see the following page.

CHAPTER -7
ALGORITHM

This chapter describes the step-by-step algorithmic procedure used in the proposed system **"From Pixels to Provenance: Harnessing Source Camera Fingerprints to Detect AI-Created Images."**

The algorithm focuses on identifying whether an input image is **camera-captured** or **AI-generated** by analyzing intrinsic source camera fingerprints.

Algorithm: AI-Generated Image Detection Using Source Camera Fingerprints

Step 1: Input Image Acquisition

- Accept a digital image from the user through the system interface.
- Validate the image format (e.g., JPG, PNG, JPEG).
- Store the image temporarily for processing.

Step 2: Image Preprocessing

- Convert the image to a standardized format and resolution.
- Apply grayscale conversion if required.
- Remove high-level content using denoising filters.
- Normalize pixel intensity values to reduce illumination variations.

Step 3: Noise Residual Extraction

- Apply a denoising algorithm to suppress image content.
- Extract the **noise residual** by subtracting the denoised image from the original image.
- Preserve sensor-level noise information required for fingerprint analysis.

Step 4: Source Camera Fingerprint (PRNU) Extraction

- Analyze the noise residual to extract **Photo-Response Non-Uniformity (PRNU)** patterns.
- Enhance fingerprint signals using high-pass filtering.
- Generate a fingerprint representation for the input image.

Step 5: Feature Vector Generation

- Convert extracted fingerprint information into numerical feature vectors.
- Select discriminative statistical and frequency-domain features.
- Prepare the feature set for classification.

Step 6: Machine Learning Classification

- Input the feature vector into a trained machine learning classifier.
- Compare fingerprint characteristics against learned camera-origin patterns.

- Determine whether the image contains authentic camera fingerprints.

Step 7: Decision Making

- If valid camera fingerprint patterns are detected:
  - Classify the image as **Camera-Captured (Real Image)**.
- Else:
  - Classify the image as **AI-Generated Image**.

Step 8: Result Generation and Display

- Generate the final classification result.
- Display the detected image origin along with confidence score (if applicable).
- Store the result for future reference (optional).

Algorithm Termination

- End the process after result display.

Algorithm Flow Summary

1. Image Upload
2. Preprocessing
3. Noise Residual Extraction
4. Camera Fingerprint (PRNU) Extraction
5. Feature Vector Creation
6. Machine Learning Classification
7. Decision & Result Display

Advantages of the Algorithm

- Uses intrinsic sensor-level characteristics
- Robust against visually realistic AI-generated images
- Less affected by post-processing operations
- Provenance-aware and future-proof

Chapter-8
SYSTEM TEST

System testing is a critical phase in the development of the proposed AI-generated image detection framework. The primary objective of testing is to ensure that the system functions correctly, meets specified requirements, and produces reliable and accurate results. Testing helps identify errors, validate system performance, and verify that individual components and integrated modules operate as expected under various conditions.

Testing is performed with the intent of discovering faults or weaknesses in the system and ensuring that the final product satisfies user expectations. Multiple testing strategies are employed to evaluate functionality, reliability, and robustness of the system.

TYPES OF TESTS

Unit Testing

Unit testing involves testing individual components or modules of the system independently to verify that they function correctly. Each module, such as image preprocessing, fingerprint extraction, and classification, is tested in isolation to ensure that it produces the expected output for valid inputs.

This type of testing is performed after the completion of each module and before system integration. Unit testing focuses on internal logic, decision paths, and error handling mechanisms. It ensures that every functional unit meets its design specifications and operates correctly under normal and edge-case conditions.

Integration Testing

Integration testing is conducted to verify the interaction between integrated modules of the system. Once individual components are tested independently, they are combined and tested as a group to ensure seamless data flow and correct interaction.

In the proposed system, integration testing validates the workflow from image upload through preprocessing, fingerprint extraction, machine learning classification, and result display. This testing helps identify issues related to data compatibility, interface mismatches, and communication errors between modules.

Functional Testing

Functional testing evaluates the system against its functional requirements to ensure that all specified features are working as intended. This testing verifies that the system accepts valid inputs, rejects invalid inputs, performs expected operations, and generates correct outputs.

Functional testing focuses on key system operations such as image upload, preprocessing, classification, and result visualization. It ensures that user interactions and system responses align with documented requirements and use cases.

System Testing

System testing is performed on the complete and fully integrated system to validate overall behavior and performance. It ensures that the system meets both functional and non-functional requirements under realistic operating conditions.

This testing evaluates system reliability, accuracy, response time, and stability. System testing confirms that the image detection framework functions correctly as a whole and produces consistent results across different input images.

White Box Testing

White box testing is a testing approach in which the internal structure, logic, and code of the system are examined. Test cases are designed based on knowledge of the system's internal workings, including control flow, data flow, and decision points.

This testing is particularly useful for validating algorithm implementation, noise extraction logic, and machine learning workflows. White box testing ensures that internal operations are optimized and free from logical errors.

Black Box Testing

Black box testing evaluates the system without any knowledge of its internal implementation. The system is treated as a black box, and testing is performed by providing inputs and analyzing outputs.

This testing focuses on validating system behavior from the user's perspective. It ensures that the system responds correctly to various inputs and produces expected results without considering how those results are internally generated.

Test Strategy and Approach

The testing strategy combines manual and automated testing techniques to ensure comprehensive system evaluation. Test cases are designed based on system requirements, use cases, and operational scenarios. Both normal and abnormal input conditions are considered to evaluate system robustness.

Test Objectives

The main objectives of system testing are:

- To ensure that all system modules function correctly
- To verify accurate classification of AI-generated and real images
- To confirm smooth navigation and user interaction
- To ensure timely response and result generation
- To validate system stability and reliability
- Features to Be Tested
- Image upload and validation
- Image preprocessing accuracy

- Camera fingerprint extraction
- Machine learning classification accuracy
- Result display and user feedback
- Error handling and input validation
- Test Results

All the test cases designed for the proposed system were executed successfully. The system performed as expected, and no critical defects were encountered during testing. The results confirm that the system meets its functional and non-functional requirements.

Acceptance Testing

User acceptance testing was conducted to ensure that the system meets user expectations and operational requirements. Feedback indicated that the system is easy to use, responsive, and effective in detecting AI-generated images.

Test Result:

All acceptance test cases passed successfully with no reported issues

TESTING SCENARIOS TABLE							
Test Case ID	**Test Scenario**	**Testing Type**	**Test Description**	**Input Data**	**Expected Result**	**Actual Result**	**Status**
TS-01	user Login	Unit Testing	Verify service provider login with valid credentials	Valid username & password	Login successful and dashboard displayed	As expected	Pass
TS-02	Invalid Login Attempt	Validation Testing	Verify system behavior for invalid login credentials	Invalid username/ password	Error message displayed	As expected	Pass

TS-03	Dataset Upload	Integration Testing	Verify dataset upload and storage in server	Traffic video dataset	Dataset uploaded successfully	As expected	Pass
TS-04	Train & Test Model	Integration Testing	Verify training and testing process on uploaded dataset	Labeled traffic dataset	Model trains and tests without error	As expected	Pass
TS-05	Accuracy Bar Chart Display	Output Testing	Verify graphical display of trained & tested accuracy	Trained model metrics	Bar chart displayed correctly	As expected	Pass
TS-06	View Prediction Result	System Testing	Verify accident prediction for test video	Traffic video clip	Accident / Non-Accident predicted correctly	As expected	Pass
TS-07	Accident Type Ratio Graph	Output Testing	Verify visualization of accident vs non-accident ratio	Prediction results	Ratio graph displayed correctly	As expected	Pass
TS-08	Download Predicted Dataset	System Testing	Verify downloading of prediction result file	Prediction dataset	File downloaded successfully	As expected	Pass

TS-09	Remote User Prediction	User Acceptance Testing	Verify remote user can predict accident type	Traffic input data	Correct prediction displayed	As expected	Pass
TS-10	View Remote Users List	System Testing	Verify service provider can view registered users	User database	User list displayed	As expected	Pass

CHAPTER-9
CONCLUSION & FUTURE SCOPE

CONCLUSION:

In this project, a provenance-aware forensic framework has been presented to address the growing challenge of distinguishing AI-generated images from authentic camera-captured images. With the rapid advancement of generative artificial intelligence, traditional image forensics techniques that rely on visual inspection or content-based artifacts are no longer sufficient. The proposed system shifts the focus from appearance-based detection to intrinsic source analysis by leveraging source camera fingerprints embedded during the image acquisition process.

By analyzing sensor-level artifacts such as noise residuals and camera-specific fingerprints, the system effectively differentiates real images from AI-created images that inherently lack physical sensor characteristics. The integration of image preprocessing, fingerprint extraction, and machine learning–based classification enables accurate and reliable image provenance verification. The system design emphasizes automation, robustness, and scalability, making it suitable for real-world forensic applications.

Experimental evaluation and system analysis demonstrate that the proposed approach provides improved reliability compared to conventional methods, even in the presence of post-processing operations such as compression and resizing. The framework offers a practical solution for digital image authentication and contributes toward restoring trust in digital media. Overall, the study highlights the effectiveness of source camera fingerprint analysis as a strong forensic indicator for detecting AI-generated imagery and supporting secure digital ecosystems.

FUTURE SCOPE

Although the proposed system delivers promising results, several opportunities exist to further enhance its functionality and applicability. Future work can focus on extending the framework to support video forensics by analyzing frame-level camera fingerprints and temporal consistency. Incorporating advanced deep learning architectures may further improve robustness against emerging generative models, including diffusion-based image synthesis techniques.

The system can also be expanded to handle large-scale social media platforms by integrating real-time image verification pipelines and cloud-based deployment. Another potential direction is the fusion of camera fingerprint analysis with cryptographic watermarking and metadata verification to create a multi-layered provenance assurance system. Additionally, adapting the framework to identify specific camera brands or devices could enhance forensic investigations and legal evidence validation.
In the future, the proposed system can play a significant role in digital journalism, cybercrime investigation, and misinformation mitigation by providing reliable, automated image authentication tools. Continuous updates and model retraining will ensure adaptability to evolving AI technologies, thereby strengthening long-term digital trust and media integrity.

Chapter-10
CODE

References:

[1] J. Lukas, J. Fridrich, and M. Goljan, "Digital camera identification from sensor pattern noise," IEEE Transactions on Information Forensics and Security, vol. 1, no. 2, pp. 205–214, Jun. 2006.
[2] M. Chen, J. Fridrich, M. Goljan, and J. Lukas, "Determining image origin and integrity using sensor noise," IEEE Transactions on Information Forensics and Security, vol. 3, no. 1, pp. 74–90, Mar. 2008.
[3] H. Farid, "Image forgery detection: A survey," IEEE Signal Processing Magazine, vol. 26, no. 2, pp. 16–25, Mar. 2009.
[4] A. Swaminathan, M. Wu, and K. J. R. Liu, "Digital image forensics via intrinsic fingerprints," IEEE Transactions on Information Forensics and Security, vol. 3, no. 1, pp. 101–117, Mar. 2008.
[5] J. Fridrich, "Digital image forensics," IEEE Signal Processing Magazine, vol. 26, no. 2, pp. 26–37, Mar. 2009.
[6] I. Goodfellow et al., "Generative adversarial networks," Communications of the ACM, vol. 63, no. 11, pp. 139–144, 2020.
[7] A. Rössler et al., "FaceForensics++: Learning to detect manipulated facial images," IEEE/CVF International Conference on Computer Vision (ICCV), pp. 1–11, 2019.
[8] D. Cozzolino and L. Verdoliva, "Noiseprint: A CNN-based camera model fingerprint," IEEE Transactions on Information Forensics and Security, vol. 15, pp. 144–159, 2020.
[9] S. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, "CNN-generated images are surprisingly easy to spot… for now," IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 8695–8704, 2020.

[10] Y. Zhang, F. Ding, and S. Kwong, “Detecting AI-synthesized images using noise inconsistencies,” IEEE Access, vol. 9, pp. 126453–126463, 2021.
[11] T. Karras et al., “Analyzing and improving the image quality of StyleGAN,” IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 8107–8116, 2020.
[12] X. Liu, Y. Wu, and W. Zeng, “Robust detection of GAN-generated images using frequency domain analysis,” IEEE Transactions on Multimedia, vol. 24, pp. 1231–1243, 2022.
[13] C. Guera and E. J. Delp, “Deepfake video detection using recurrent neural networks,” IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS), pp. 1–6, 2018.
[14] F. Marra, D. Gragnaniello, D. Cozzolino, and L. Verdoliva, “Detection of GAN-generated fake images over social networks,” IEEE Conference on Multimedia Information Processing and Retrieval (MIPR), pp. 384–389, 2018.
[15] N. Yu, L. Davis, and M. Fritz, “Attributing fake images to GANs: Learning and analyzing GAN fingerprints,” IEEE/CVF International Conference on Computer Vision (ICCV), pp. 7556–7566, 2019